

Reward Shaping and Opponent Curricula for Sparse-Reward Multi-Agent Play in SoccerTwos

Chuye Zhang
Georgia Institute of Technology
czhang883@gatech.edu

Jianuo Qiu
Georgia Institute of Technology
jqiu317@gatech.edu

Abstract: We apply delta-based reward shaping and a four-stage opponent curriculum to PPO in the SoccerTwos environment. Reward shaping alone bootstraps learning within 0.25M steps and reaches 8/10 wins versus the CEIA baseline; our submitted agent LUCKETS_AGENT adds a redesigned four-stage curriculum (trained over a stitched 35.8M-step three-segment chain) and beats the shaping-only variant 7/10 in head-to-head play. Pure PPO self-play without shaping scores zero goals in 10 matches, confirming that the reward signal—not the self-play structure—is the primary bottleneck. We also report a first-attempt curriculum that failed to advance past Stage 0 due to an opponent-policy mapping bug, which directly motivated the redesign.

Code: <https://github.com/ZhangChuye/soccer-twos-starter-18>

Submitted agent (single agent, covers all rubric criteria): LUCKETS_AGENT — Policy performance, Reward modification, Curriculum learning.

Keywords: Multi-agent RL, reward shaping, curriculum learning, PPO, SoccerTwos

1 Overview

Raw PPO on SoccerTwos faces two obstacles: (i) goals are rare, so the sparse ± 1 reward gives almost no gradient signal in early training; and (ii) a self-play opponent that is always “yourself” produces a non-stationary target that offers little stable learning signal once both sides converge on a trivial strategy. We address (i) with small delta-based dense reward terms that fire on every step without changing the optimal policy, and (ii) with a four-stage opponent curriculum that starts against a random opponent and gradually escalates to the CEIA baseline and then self-play [1]. We run three configurations—reward shaping only, pure sparse self-play, and the combined agent—to isolate each contribution.

2 Method

Algorithm and library. All runs use Ray RLLib [2] with the PPO trainer [3] (Ray 1.4.0, PyTorch, two-layer MLP [256, 256], ReLU). Hyperparameters: Appendix A.

Reward modification. The default environment returns ± 1 only at goals plus a small per-step penalty. We wrap it with `RewardShaperWrapper` (`utils.py`) which adds two dense terms each step:

$$r_t^{\text{prox}}(i) = \alpha_{\text{prox}}(d_{t-1}(i) - d_t(i)), \quad r_t^{\text{prog}}(i) = \alpha_{\text{prog}}(x_t^{\text{ball}} - x_{t-1}^{\text{ball}}) \text{sign}(\text{team}(i)) \quad (1)$$

where $d_t(i) = \|p_t^{\text{player}}(i) - p_t^{\text{ball}}\|_2$ and x^{ball} is the ball’s forward coordinate; $\alpha_{\text{prox}}=0.02$, $\alpha_{\text{prog}}=0.05$. Both terms are deltas: agents are rewarded only for approaching the ball and

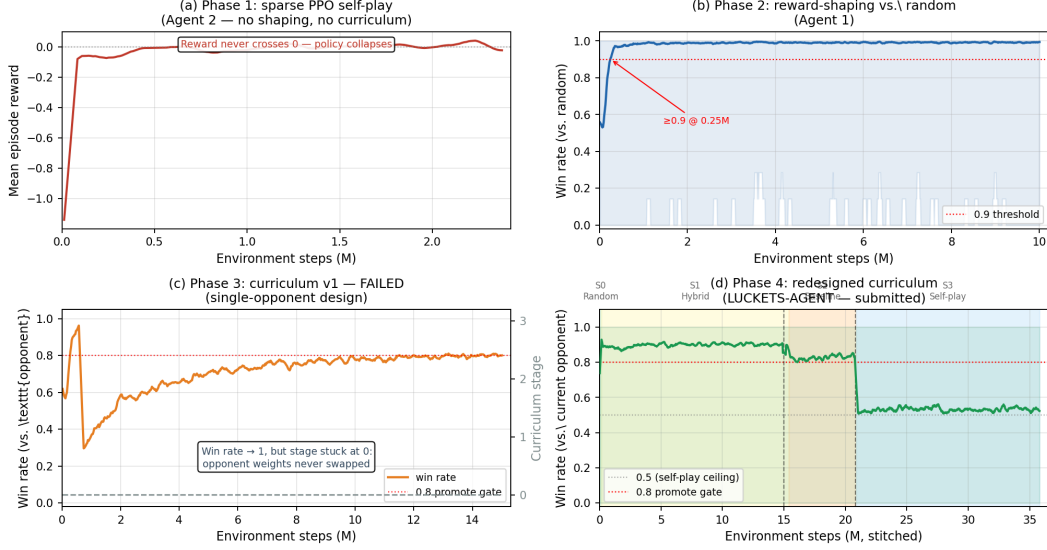


Figure 1: Training curves for all four experiments. (a) Phase 1, sparse self-play (67194, 2.4M steps): episode reward never crosses 0. (b) Phase 2, reward shaping vs. random (a6c6d, 10M steps): win rate ≥ 0.9 within 0.25M steps, saturating near 1.0. (c) Phase 3, curriculum v1 — failed (7b8e4, 15M steps): win rate climbs but curriculum stage remains 0 throughout (right axis) because the single-opponent design cannot swap weights at runtime. (d) Phase 4, re-designed curriculum (c33b8 + 6eb9a + 02fdb, 35.8M steps stitched): colored backgrounds mark the four curriculum stages; black dashed verticals are the two segment resumes; red dotted is the 0.8 promotion gate; grey dotted is the 0.5 self-play symmetric ceiling.

moving it toward the opponent goal, not for already being close—so the shaping is nearly potential-based [4], preserving the optimal policy. Motivation: the sparse goal reward fires on $< 1\%$ of steps, so PPO’s gradient is dominated by advantage variance; per-step deltas bootstrap the agent past the “nothing happens” phase efficiently.

Opponent curriculum (LUCKETS_AGENT). A `CurriculumCallback` remaps orange-team policies in four stages: (0) Random: both $\rightarrow$ `opp_rand`; (1) Hybrid: agent 2 $\rightarrow$ `opp_base` (CEIA), agent 3 $\rightarrow$ `opp_rand`; (2) Baseline: both $\rightarrow$ `opp_base`; (3) Self-play: both $\rightarrow$ `opp_self` (frozen learner snapshot). Promotion requires win rate ≥ 0.80 for 3 consecutive iterations. The current stage is persisted to `.curriculum_stage` and read by every Ray worker on each rollout. Motivation: a policy trained only against a random opponent has a narrow occupancy distribution; directly switching to the CEIA baseline collapses training. The ramp keeps the learning problem just difficult enough to make progress at each stage.

3 Experimental Results

Exploration path (Fig. 1). Phase 1 (Agent 2): sparse PPO self-play (67194, 2.4M steps) scored zero goals. Phase 2 (Agent 1): adding shaping vs. a frozen random opponent (a6c6d, 10M steps) crossed 0.9 win rate at 0.25M and reached 8/10 vs. CEIA. Phase 3: a first-attempt curriculum with a single shared opponent policy (7b8e4, 15M steps) hit 0.99 win rate but stayed at Stage 0 throughout—our promotion gate fired but the single-policy design could not be remapped at runtime in RLlib’s worker config. Phase 4 (LUCKETS_AGENT): we redesigned the curriculum with three separate opponent policies (`opp_rand`, `opp_base`, `opp_self`) selected by `policy_mapping_fn` from a stage file persisted across Ray workers, and trained three resumed segments: Stage 0 $\rightarrow$ 1 (0–15M, c33b8), Stage 1 $\rightarrow$ 2 (15–20.8M, 6eb9a), Stage 2 $\rightarrow$ 3 (20.8–35.8M, 02fdb). `checkpoint-600` of 02fdb ships as LUCKETS_AGENT.

	Ex. player	CEIA	Ag. 1 (rew. shaped)	Ag. 2 (self-play)
Agent 1: reward-shaped	10/0/0	8/0/2	—	10/0/0
Agent 2: self-play	0/0/10	0/10/0	—	—
Agent 3: LUCKETS (ours)	10/0/0	8/2/0	7/1/2	10/0/0

Table 1: Head-to-head record (wins / losses / draws), 10 matches per pairing. Each row is the blue team; column is the orange opponent. “—” = symmetric duplicate not run.



Figure 2: LUCKETS_AGENT (blue) vs. CEIA baseline (orange) during evaluation; score 5–1 mid-game.

Head-to-head evaluation. We ran 10 headless matches per pairing (1,000 step timeout, `explore=False`). Table 1 reports win/loss/draw from team 0’s perspective. Agent 3 (LUCKETS) wins 8/10 versus CEIA (2 losses, 0 draws) and beats Agent 1 7-1-2. Agent 1 also wins 8/10 versus CEIA but with 2 draws instead of losses, indicating Agent 3 is more aggressive but occasionally over-commits. Agent 2 draws all 10 matches against the example player and loses all 10 to CEIA.

4 Analysis and Discussion

Reward shaping. Result: higher reward and faster convergence. With sparse reward, the policy gradient is dominated by variance because goal events fire on $< 1\%$ of steps. The delta form of Eq. 1 converts every step into a small informative signal without changing the optimal policy, and using deltas (not absolute values) avoids the ball-chasing attractor an absolute-distance reward would create. This is why Agent 1 reaches 0.9 win rate within 0.25M steps while Agent 2 (sparse) never scores.

Curriculum. Result: higher peak performance. Agent 3 beats Agent 1 7-1-2 despite using identical shaping, because a policy trained only against random opponents has a narrow occupancy distribution that is partly off-support against CEIA defense. Stages 1–2 shift training toward that distribution, Stage 3 keeps it expanding. Fig. 1d shows the expected staircase: win rate drops at each stage advance and recovers. Curriculum v1 failed for an architectural reason: with a single shared `opponent` policy, weight swaps mid-run require copying tensors through every Ray worker, but our naive callback only updated the driver’s copy. The fix is to declare all three opponent policies up front and route each agent ID through `policy_mapping_fn` from a worker-shared stage file.

Pure self-play failure. Without shaping, no player generates a scoring trajectory early enough to provide a gradient; both sides converge on a trivial idle strategy with zero goals, entropy collapses, and rotating snapshots cannot perturb the fixed point because the reward signal for escaping it is identically zero.

Limitations. Both agents stop at 8/10 vs. CEIA, not a decisive 9/10; the two Agent 3 losses suggest over-committing on offense. Shaping ignores teammate position, so no passing incentive is learned. A symmetric demotion rule (promote at ≥ 0.8 , demote at ≤ 0.4) is the most likely next improvement.

Acknowledgments

We thank the course staff for the SoccerTwos starter kit [5] and the CEIA baseline, and the Georgia Tech PACE team for cluster access.

References

- [1] Y. Bengio, J. Louradour, R. Collobert, and J. Weston. Curriculum learning. In International Conference on Machine Learning (ICML), 2009.
- [2] E. Liang, R. Liaw, R. Nishihara, P. Moritz, R. Fox, K. Goldberg, J. Gonzalez, M. Jordan, and I. Stoica. RLlib: Abstractions for distributed reinforcement learning. In International Conference on Machine Learning (ICML), 2018.
- [3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.
- [4] A. Y. Ng, D. Harada, and S. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In International Conference on Machine Learning (ICML), 1999.
- [5] B. Oliveira. Soccer-twos: a starter kit for Unity ML-Agents SoccerTwos. <https://github.com/bryanoliveira/soccer-twos-env>, 2021.

A Hyperparameters

Hyperparameter	Value
Algorithm	PPO (Ray RLlib 1.4.0, PyTorch)
Policy/value network	MLP [256, 256], ReLU, shared trunk + value head
Learning rate	3×10^{-4}
Discount γ	0.995
GAE λ	0.95
PPO clip ϵ	0.2
Entropy coefficient	0.01
Value-loss coefficient	0.5
SGD minibatch / epochs	512 / 10
Train batch size	12,000
Rollout fragment	1,000
Workers $\times$ envs/wkr	$12 \times 3 = 36$
Total env steps	$\sim 10\text{M}$ (Agent 1); $\sim 2.4\text{M}$ (Agent 2); $\sim 35.8\text{M}$ (Agent 3)
Shaping α_{prox} (Agents 1, 3)	0.02
Shaping α_{prog} (Agents 1, 3)	0.05
Promotion gate (Agent 3)	win rate ≥ 0.80 for 3 consecutive iters

Table 2: Final-run hyperparameters. Shaping coefficients are zero for Agent 2.